AE5VG

AMATEUR RADIO STATION

# The Operating Skills

A field guide to the operating skills for **fldigi-mcp** and **n3fjp-mcp** — how an agent keys the transmitter and returns to receive at the exact instant an over ends, runs a contest QSO from CQ to logged contact, and survives everything a real band throws at it. Every rule in this guide was proven on the air.

FIELD GUIDE · VERSION 0.4 · SEPTEMBER 2026

| 3 | 6 | 7 | 3 |
|---|---|---|---|
| OPERATING SKILLS | RULES IN THE STANDARD | SPECIAL CASES HANDLED | WORKED EXAMPLES FROM THE AIR |

© 2026 STEFAN BRUNNER · AE5VG

FIELD-PROVEN AT ARRL FIELD DAY 2026 · CLASS 2A · BPSK31

# Contents

BEFORE YOU START

A Skills at a glance

B Installing the skills

C Your first session — Claude for hams

D How the system works

E The operating standard

THE SKILLS

01 fldigi-operating

02 contest-operating

03 signal-hunting

ON THE AIR

F The special-case playbook

G Worked examples from Field Day

REFERENCE

H Logging to N3FJP

I About & provenance

CHAPTER A · BEFORE YOU START

# Skills at a Glance

Three skills, one operating system. Each in one view — what it governs, and what you get. One full page per skill inside.

| # | SKILL | SHIPS WITH | WHAT IT GOVERNS | WHAT YOU GET |
|---|---|---|---|---|
| 01 | **fldigi-operating** | `fldigi-mcp` | The radio: keying the transmitter, returning to receive the instant an over ends, and polling the RX stream so no caller is missed. | Instant TX→RX handoff + a caller never ignored |
| 02 | **contest-operating** | `n3fjp-mcp` | The contest: the QSO state machine from CQ to logged contact, the special-case playbook, and verified N3FJP logging. | Completed exchanges + a clean, duplicate-free log |
| 03 | **signal-hunting** | `fldigi-mcp` | The band: naming each signal's mode from the audio, ranking the station that sits still and calls CQ, tuning to it, and reading fldigi's Signal Browser where the proposed `browser.*` patch is present. | The right station found, named and tuned before you call |

> WHO READS THESE SKILLS
>
> You don't invoke these skills by name — the **agent** does, when you say things like *“call CQ on 20 meters”* or *“run the Field Day loop.”* The skills are the agent's operating procedures. This guide is for the human at the station: what the agent will do on your behalf, and how to verify it is doing it right.

# Installing the Skills

The skills and the servers install separately. A desktop connector ( `.mcpb` ) installs only its server, so you add the skills yourself. You do this once.

### In the Claude desktop app

1. **Download the files.**
   On the Downloads page at ae5vg.com, under fldigi-mcp, click `fldigi-mcp.mcpb` , `fldigi-operating.skill` and `signal-hunting.skill` . Under n3fjp-mcp, click `n3fjp-mcp.mcpb` and `contest-operating.skill` .

2. **Install the connectors.**
   In Claude, open Settings → Desktop app, click Manage next to Extensions, and drag `fldigi-mcp.mcpb` onto that page. If dragging does not work, click Advanced settings, then Install extension, and choose the file. The new connector then shows under Settings → Customize → Connectors. Then do the same with `n3fjp-mcp.mcpb` .

3. **Set the transmit gate.**
   In the fldigi-mcp settings, type your callsign in Callsign. With no callsign set, the station is receive-only and nothing can key the radio.

4. **Add the skills.**
   In Claude, open Customize → Skills. Click Add, then Upload skill, and choose `fldigi-operating.skill` , `signal-hunting.skill` and `contest-operating.skill` all at once (hold ⌘ on a Mac or Ctrl on Windows to select several). Click Upload.

Cowork in the Claude desktop app uses the same connectors and skills. Nothing else to install.

### For advanced users: Claude Code in a terminal, or another MCP client

1. **Copy the skill directories**
   from each repo into the client's skills folder, for Claude Code `~/.claude/skills/` or a project's `.claude/skills/` : `skills/fldigi-operating/` , `skills/signal-hunting/` and `skills/contest-operating/` .

2. **Connect the MCP servers**
   ( `uvx fldigi-mcp` , `uvx n3fjp-mcp` or from the repos) so the tools the skills reference are available.

The MCP servers and skills are not tied to Claude. MCP is an open standard, so other assistants and agentic harnesses such as OpenClaw should work too. They have not been tested yet.

### Verify `thirty seconds`

1. **Check the radio link.**
   Ask *“what's the fldigi status?”* — mode, frequency, and T/R state should come back. If not, the `diagnostics` tool tells you whether it's a network problem before you blame fldigi.

2. **Dry-run the handoff.**
   On a dummy load: *“send TEST DE <YOURCALL> and return to receive.”* fldigi should key, send, and drop PTT on its own — no lingering carrier.

GOOD TO KNOW

fldigi's XML-RPC default port is **7362**. A different configured port (e.g. 7372) usually means a localhost→remote forwarder such as `mcp-host-bridge` is intended — check `FLDIGI_PORT` before concluding fldigi is down.

# Your First Session — Claude for Hams

You do not need to know anything about AI to run this station. If you can install fldigi and type a sentence, you can operate. This chapter is the whole learning curve.

## What you're actually installing

**Claude** is an assistant made by Anthropic. It runs in an ordinary desktop program — the **Claude Desktop app** (Windows or Mac), which looks like a messaging window: you type at the bottom, answers appear above. The two **MCP servers** in this project are best thought of as rig-interface cables: one plugs Claude into fldigi, the other into N3FJP. You type plain English; Claude works the programs. The **skills** are the operating procedure Claude follows: when to transmit, how an exchange runs and when to log. Nothing about your fldigi or N3FJP setup changes — they run exactly as they always have.

## Setup, once

1. **Install the Claude Desktop app.**
   Download it from `claude.ai/download`, install like any program, and sign in (create the account on the same page). A paid plan is recommended — the free tier can run out of messages mid-QSO.

2. **Run your shack software as usual.**
   Start fldigi (its XML-RPC interface is on by default). If you want contest logging, start N3FJP and enable its TCP API (Settings → Application Program Interface → checked). fldigi must key the radio the usual way, under Configure → Config Dialog → Rig Control: Hardware PTT for an interface such as a RigBlaster, nothing for a VOX interface such as a SignaLink. If fldigi's T/R button keys the radio, Claude can too. The N3FJP loggers run only on Windows. On a Mac, run N3FJP in a Windows virtual machine or on a Windows PC and connect it with `mcp-host-bridge`, as *Where everything runs* below shows.

3. **Add the two connectors.**
   Install `fldigi-mcp.mcpb` and `n3fjp-mcp.mcpb` as in Chapter B, and enter your callsign in the fldigi settings. That callsign is the transmit gate.

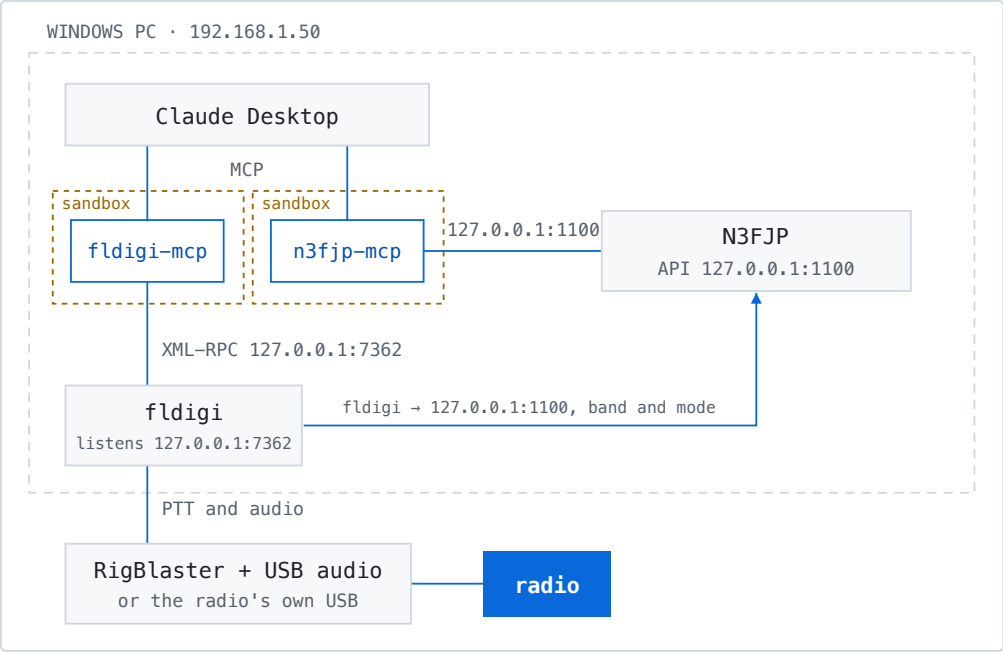
4. **Add the three skills.**
   Upload `fldigi-operating.skill`, `signal-hunting.skill` and `contest-operating.skill` under Customize → Skills, as in Chapter B. You only do this once.

5. **Say hello.**
   Open a new chat and choose **Sonnet** with the effort set to **Medium** in the model menu under the box where you type. That is the recommended setting: it answers quickly enough to keep up with a QSO, and it uses less of your plan's usage, so a long session costs less. Then type: *"What's the fldigi status?"* If mode, frequency, and T/R state come back, your station is connected.

## Where everything runs

The N3FJP loggers run only on Windows. fldigi runs on Mac, Windows and Linux. That gives two common stations, shown here with example addresses. Each MCP server runs in a sandbox that can reach only 127.0.0.1.

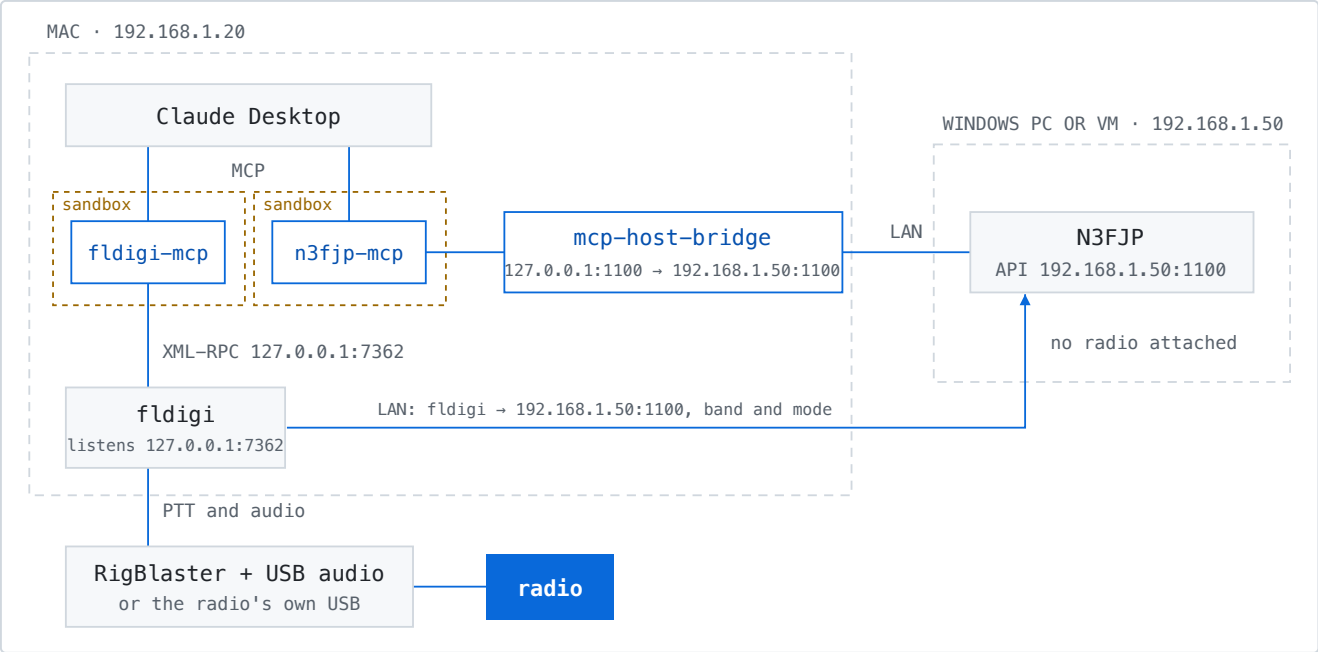
**All on one Windows PC.** Claude, fldigi, N3FJP and the radio are on one PC. Everything talks over 127.0.0.1, and no bridge is needed.




One Windows PC. Every connection stays on 127.0.0.1.

| SOCKET | COMPUTER | LISTENING | WHO CONNECTS |
|---|---|---|---|
| 127.0.0.1:7362 | Windows | fldigi XML-RPC | fldigi-mcp, from inside its sandbox |
| 127.0.0.1:1100 | Windows | N3FJP TCP API | n3fjp-mcp, from inside its sandbox; fldigi, to set band and mode |

**Claude and fldigi on a Mac, N3FJP on Windows.** The radio stays on the Mac, connected by USB or through a RigBlaster and a USB audio adapter. N3FJP runs in a Windows virtual machine or on a Windows PC, with no radio attached. n3fjp-mcp reaches it through mcp-host-bridge on the Mac: `mcp-host-bridge install n3fjp --to 192.168.1.50`. Two connections cross the network, both to N3FJP's API on port 1100.




Mac and Windows. The bridge carries n3fjp-mcp to N3FJP; fldigi connects to N3FJP directly.

| SOCKET | COMPUTER | LISTENING | WHO CONNECTS |
|---|---|---|---|
| 127.0.0.1:7362 | Mac | fldigi XML-RPC | fldigi-mcp, from inside its sandbox |
| 127.0.0.1:1100 | Mac | mcp-host-bridge | n3fjp-mcp, from inside its sandbox |
| 192.168.1.50:1100 | Windows | N3FJP TCP API | mcp-host-bridge, over the LAN; fldigi, over the LAN, to set band and mode |

**Band and mode in N3FJP come from fldigi.** In N3FJP, open Settings → Rig Interface (Ctrl+R), choose N3FJP API under Select Rig, enter the Windows computer's name and port 1100 under Command to Read Frequency (for example `MYPC:1100`), leave Mode Determined By at Rig, and click Save Settings. In fldigi, open Configure → Config Dialog → Logging → N3FJP logs, enter N3FJP's address (127.0.0.1 on one PC, the Windows address from a Mac) and port 1100, and tick Connect to the N3FJP log server.

### Operating is a conversation

There are no commands to memorize and no syntax to get wrong. You type what you would tell a competent operator sitting at your rig:

| YOU TYPE | WHAT HAPPENS |
|---|---|
| `What's the fldigi status?` | Mode, frequency, T/R, and callsign gate — your pre-flight check. |
| `Set the rig to 14.070 USB and BPSK31.` | Claude QSYs the rig and sets the modem. |
| `Call CQ Field Day until someone answers. Work them — our exchange is 2A STX — log the contact to N3FJP, then keep calling.` | The full loop in this guide: CQ, listen, exchange, TU, log, repeat. Claude narrates each QSO as it happens. |
| `stop` | Transmission stops immediately and the radio returns to receive. Works at any moment, mid-over included. |

YOU ARE STILL THE CONTROL OPERATOR

Claude operates; **you** hold the license. Part 97 responsibility — supervision, station identification, staying in the band — remains yours, exactly as if a guest operator sat at your key. Stay at the station while it transmits. Two safeguards are built in: with no callsign configured the station physically cannot key up, and “stop” always works.

FIRST TIME? DUMMY LOAD, THEN THE AIR.

Start on a dummy load and test only the transmit side: *“call CQ once, then listen”*. The rig should key, send and drop back to receive, with no carrier left on. Nobody can answer a dummy load, so there is no QSO and no log entry yet. Then connect the antenna, start the loop, and watch the first contact all the way into the log. If anything misbehaves, type *“run the fldigi diagnostics”* and read the answer; it distinguishes a network problem from an fldigi problem.

CHAPTER D · BEFORE YOU START

# How the System Works

A QSO is a loop with four states. **fldigi-operating** governs the radio in every state; **contest-operating** decides what to say and when to log.

**1 · CQ**

*Anyone out there?*

```
CQ FD CQ FD de <MYCALL>
<MYCALL> pse k
```

**2 · EXCHANGE**

*Caller decoded — send ours*

```
<CALL> de <MYCALL>
Pse copy 2A 2A STX STX
de <MYCALL> BK
```

**3 · TU**

*They QSL — confirm theirs back*

```
QSL <CLASS> <SECT> TU
<CALL> de <MYCALL> GL
FD sk sk
```

**4 · LOG**

*Write it, then QRZ*

```
log → log_qso … check
logged: true
```

- **Skills trigger on plain language.** No commands to memorize. Say *"call CQ until someone answers, then work them and log it"* and both skills engage. You can also ask for a skill by name when you want a specific lens.
- **The radio layer runs first because it changes everything above it.** An over that doesn't drop PTT on time transmits idle carrier on top of the answering station — no exchange strategy survives that. The `^r` handoff is not optional.
- **The two skills chain deliberately.** fldigi-operating delivers clean overs and an honestly-read RX stream; contest-operating consumes that stream, runs the exchange, and hands a confirmed QSO to the logger. Findings that die on a garbled callsign become an AGN request instead of a log entry.
- **The operator stays in command.** Say "stop" at any time — `transmit → abort` drops PTT immediately and discards queued text. The callsign gate means an unconfigured station cannot transmit at all.

> THE HABIT THAT MATTERS
>
> Watch the loop once before you trust it. A dummy load only proves the transmit side: the CQ, the drop back to receive, and "stop". The rest needs a real station, so on the air watch the first complete QSO: TX drop, listen window, exchange, log entry.

# The Operating Standard

Six rules, inherited by both skills. Every one of them exists because we broke it once — on the air, at Field Day 2026.

1. **Never poll the TX buffer to decide when to stop.**
   fldigi holds PTT on an empty buffer by design. End every over with `^r` (via `transmit → send`) and fldigi drops to receive at the exact sample the last character finishes. Polling always lags; the lag is transmitted on top of your caller.

2. **The RX buffer never contains your own echo.**
   Every character came from an external station. Your callsign in RX means someone is calling you — pursue it. "Dismiss as own echo" logic makes you ignore callers; we wrote it, it did, and we deleted it.

3. **Never log without a confirmed callsign.**
   Class and section can be inferred from a repeat; a callsign cannot. An incomplete QSO is not a contact — abandon it and CQ again.

4. **Trust `logged: true`, nothing else.**
   N3FJP's ENTERRESPONSE can report 0 for a QSO that was written (networked mode commits asynchronously). Blind retries on `records_added: 0` produced duplicate log records. Retry only when `logged` is false.

5. **Budget two overs for an unreadable station.**
   One AGN, one listen window. Still garble? Back to CQ. A marginal signal that costs five minutes is four QSOs you didn't make.

6. **Double what must survive QRM.**
   `2A 2A STX STX`, callsign twice on CQ, and echo the other station's exchange back in your TU so they can verify what you logged before you QRZ.

> WHY THIS EXISTS
>
> These skills were built by running one real autonomous Field Day session and keeping the after-action report — the full account is in `n3fjp-mcp/docs/LESSONS-FIELD-DAY-2026.md`, and the worked examples in chapter G retell it verbatim.

SKILL 01 · SHIPS WITH FLDIGI-MCP

# fldigi-operating

Governs the radio: transmit, hand off to receive, listen properly.

### Transmitting an over

```
transmit → send "<over text>\n"   (return_to_rx=true, the default)
```

One call: clears the TX buffer, appends `^r`, keys the radio. End messages with a real newline (`\n`) — the two-character string `^n` transmits literally. If building an over incrementally with `text → add_tx`, the last fragment must end in `^r`, with nothing added after it.

| OPERATION | BEHAVIOR | USE FOR |
|---|---|---|
| **`^r` via `transmit → send`** | Graceful auto-return the moment the over ends | Every normal over |
| **`transmit → abort`** | Immediate stop, discards queued text | Panic button — caller back early, wrong message, operator says stop |
| **`transmit → rx`** | Returns to RX only *after* the buffer drains | Almost never — slow, and *not* the stop button |

### Listening

- **Delta reads only.** `text → rx_length` for the end position, then `text → read` from your last position. Save the new position.
- **Restart detection.** `rx_length` smaller than your last position means fldigi restarted — re-baseline and continue.
- **Listen window.** After `get_trx_status` returns `"rx"`, listen ~10 seconds (1–2 polls) before the next CQ. PSK31 operators type slowly.
- **Validate callsigns** against `[A-Z0-9]{1,3}[0-9][A-Z]{1,3}` before any directed reply — QRM garbage looks call-ish.

```
loop:
  transmit → send "CQ CQ de <MYCALL> <MYCALL> pse k\n"
  poll controls → get_trx_status until "rx"
  listen ~10 s  (1–2 polls of text → rx_length)
  if plausible callsign in new RX text → work the station
  else → loop
```

SKILL 02 · SHIPS WITH N3FJP-MCP

# contest-operating

Governs the contest: what to say in each state, when a QSO is real, and when it goes in the log.

### The exchange, tuned on the air

- **Double everything important.** `2A 2A STX STX` copies far better than `2A STX` through QSB and QRM.
- **Echo their exchange back in the TU** ( `QSL 1D AZ TU W7XYZ …` ) — the other station verifies you logged them correctly before you QRZ.
- **Callsign-only callers are normal.** Some operators send just their call and wait. Send your exchange; theirs arrives in the next over. Others send everything up front — then you go straight to TU after your exchange.
- **Log while the TU is still transmitting.** The radio is busy ~15 seconds — exactly the time logging takes. When TX drops you are ready to CQ.

### Logging in one call

```
log → log_qso  {"call": "W7XYZ", "contest": "field_day",
                "exchange": {"class": "1D", "section": "AZ"}}
```

Exchange fields go in the `exchange` dict — they are **not** top-level parameters. Success is `logged: true` in the response; chapter H has the full protocol and its quirks.

> THE RULE THAT KEEPS THE LOG CLEAN
>
> Never retry on `records_added: 0` alone — that value can be 0 for a QSO that was written. Retry only when `logged` is false. Blind retries are where duplicate log records come from.

SKILL 03 · SHIPS WITH FLDIGI-MCP

# signal-hunting

Finds a station worth working the way an operator reads the waterfall: name what is there, rank the one that sits still and calls CQ, tune to it, confirm in twenty seconds.

### Two ways to hunt

The preferred way reads fldigi's own Signal Browser. It needs no audio setup, but it needs a patched fldigi (see *With the Signal Browser patch* below, proposed and not yet in fldigi). Until the patch is in fldigi, use the **sound bridge**: `signal_hunt` listens to the same receiver audio as fldigi. With the radio on a sound card cable (an iMic, a SignaLink, the radio's USB port), set `Audio input device` to the device fldigi uses, because two programs can read one input at once. When the receiver audio plays on the computer (an SDR program, a web receiver, wfview), split it with a virtual sound card: BlackHole 2ch and a Multi-Output Device on a Mac, or VB-Audio Virtual Cable on Windows. ae5vg.com, Getting Started, has the steps.

### What the tool sees

fldigi's XML-RPC exposes no spectrum, so `signal_hunt` taps the audio fldigi is listening to (the `Audio input device` setting names it) and returns ranked candidates. It needs the optional extra `fldigi-mcp[hunt]`, numpy and sounddevice. Each candidate is where the cursor goes, what modem to run, and why it ranks where it does.

| FIELD | MEANING |
|---|---|
| `carrier_hz` | The audio carrier: where the waterfall cursor goes. fldigi's Freq is dial + carrier. |
| `mode, fldigi_modem` | RTTY with its shift, CW, BPSK31/63/125, Olivia with tones and bandwidth ( `OLIVIA-8/250` ), MFSK16, DominoEX, MT63, or *unknown*. |
| `db_over_floor` | Strength, from a max-hold spectrum so a station that transmits for part of the window still shows at full strength. |
| `persistence` | Fraction of the window the signal was present at that carrier. A CQ loop is near 1; the other side of a QSO comes and goes. |
| `periodicity` | How strongly the on/off pattern repeats at lags of 8 to 60 seconds. A CQ loop repeats; a QSO in progress does not. |
| `score` | Strength weighted by persistence and periodicity. The CQing station wins. |

### With the Signal Browser patch `proposed, not yet in fldigi`

> PROPOSED API, NOT IN ANY RELEASED FLDIGI
>
> The `browser` tool and `signal_hunt method="browser"` call two XML-RPC methods that stock fldigi does not have: `browser.get_channels` (one struct per channel: `channel, freq, active, text`) and `browser.clear`. They come from a four-file patch offered to Dave W1HKJ on 11 September 2026 (w1hkj/fldigi issue 55) and shipped in fldigi-mcp under `patches/`. Until he merges it, the names and fields are a proposal and may change. On an unpatched fldigi both calls answer with a hint and nothing else in this chapter changes.

fldigi's own Signal Browser, the left-hand panel, is a bank of thirty demodulators with DCD; it copies PSK, RTTY and CW stations too faint for any spectrum to rank. On a build that carries the patch, `browser channels` or `signal_hunt method="browser"` returns every channel with its carrier and the text already copied, line breaks kept: the confirm step is done before you tune. Four synthetic PSK31 stations from 0 to −26 dB: all four copied in full. The browser names no mode; hunt the audio first when the band's mode is unknown. Building it yourself is three lines, in the README.

### Letting the station name its own mode `proposed, not yet in fldigi`

> PROPOSED API, NOT IN ANY RELEASED FLDIGI
>
> The `rsid` tool calls `rsid.get_hits` and `rsid.clear`, from a second four-file patch offered to Dave W1HKJ on 16 September 2026 (w1hkj/fldigi issue 55) and shipped in fldigi-mcp under `patches/`, on top of the browser patch. Until it is merged the names and fields are a proposal. On an unpatched fldigi the tool answers with a hint and nothing else in this chapter changes.

Naming a mode from its shape is guesswork for the hopping modes: Olivia and Contestia share a grid, so do THOR and DominoEX. Most operators on those modes send an RSID burst at the start of an over, and fldigi's detector hears it across the whole passband. Set RxID to `notify only` and the detector reports the mode without switching the modem, so the pass you are running keeps its browser. `rsid hits` then gives `{utc, mode, hz}` for every burst: the mode name is exactly what `tune_to` takes, and the frequency is where the cursor goes. Ask the band before you guess at it, especially when the analyser says *unknown* on a strong signal.

### How a mode is named

From the signal's shape, the way you would from the waterfall, with the numbers checked against the Signal Identification Wiki. Two narrow lines a standard shift apart (170 Hz in amateur use) is RTTY, carrier at the midpoint. One 30 Hz line is PSK31, unless it is keyed on and off with gaps of 40 ms or more, which makes it CW. The hopping modes show as a grid of discrete dots when you look at where the energy is from one 30 ms frame to the next: Olivia, with tones = bandwidth / spacing; MFSK16, 16 tones at 15.6 Hz; DominoEX and THOR, 18 tones. MT63 is a flat continuous block. Contestia looks like Olivia and THOR like DominoEX; only RSID separates those.

### The loop

```
signal_hunt seconds=20 (contest: mode="RTTY", seconds=40)
read candidates highest score first
tune_to carrier_hz, fldigi_modem   (RxID off during a classified pass)
wait 20 s, then text → read
if CQ and a callsign-shaped token, twice → hand to fldigi-operating
elif modem → get_quality below 30 → next candidate
else a QSO in progress → wait for SK or 73
```

> LIMITS, STATED
>
> Verified on five recordings of known mode, PSK31, CW, RTTY at 170 and 450 Hz, Olivia 8/250: 5 of 5. A small set; treat *unknown* as honest. Symbol rate is not measured yet, which is what would separate Contestia from Olivia without RSID. Without audio access the tool falls back to stepping fldigi's own `search_up` and reading `get_quality`: slow, and blind to mode. Receive only; nothing in this skill keys the transmitter.

CHAPTER F · ON THE AIR

# The Special-Case Playbook

All seven encountered live at Field Day 2026 — none of them hypothetical.

| SITUATION | WHAT THE AGENT DOES |
|---|---|
| **Callsign only, no exchange** | Normal. Send your exchange; theirs arrives in the next over. |
| **Exchange arrives with the first call** | Also normal ( `K6ABC de W7XYZ 1D AZ 1D AZ pse k` ). Send yours; they QSL; straight to TU. |
| **“No copy” / “repeat”** | Resend once, abbreviated: `<CALL> de <MYCALL> · 2A STX 2A STX · BK` . Still nothing → QRZ once, then CQ. |
| **Garbled / partial callsign** | `de <MYCALL> AGN AGN pse k` , one listen window. Still unreadable → CQ. Two overs maximum. |
| **Two stations doubled** | Interleaved callsigns and control garbage. One AGN, then CQ — usually one calls again alone. |
| **Another station CQing on frequency** | PSK's narrow passband decodes neighbors. `CQ … de <OTHERCALL>` is not a caller — ignore and continue. |
| **Signal lost mid-QSO** | Exchange never completed → do not log. An incomplete QSO is not a contact. |

> FIELD NOTE · N3FJP MODAL DIALOGS
>
> Modal dialogs on the logging PC block the entire N3FJP API. If logging calls stop responding mid-contest, a dialog is open and needs human attention — nothing the agent can clear remotely.

# Worked Examples from Field Day

Three real QSOs from the session log, 14.070 MHz BPSK31 — transcribed as decoded, garble included. All callsigns are anonymized (ARRL-style sample calls) out of respect for the stations worked; the exchanges, flow, and garble are verbatim.

### Example 1 · W7XYZ — the textbook QSO

LOGGED · 1D AZ

**TX** `CQ FD CQ FD de K6ABC K6ABC pse k`

**RX** `K6ABC de W7XYZ 1D AZ; 1D AZ  PSE K`

*Caller sent callsign and full exchange up front — no need to ask.*

**TX** `W7XYZ de K6ABC K6ABC · Pse copy 2A 2A STX STX · de K6ABC BK`

**RX** `K6ABC QSL TU 73 de W7XYZ · GL es 73 · sk`

**TX** `QSL 1D AZ TU W7XYZ de K6ABC GL FD sk sk`

*Logged while the TU was still transmitting: log_qso → logged: true.*

✓ **LOGGED** — clean copy both ways, exchange confirmed in both directions, one pass through the state machine.

### Example 2 · N4DEF — callsign first, exchange after

LOGGED · 1D NFL

**TX** `CQ FD CQ FD de K6ABC K6ABC pse k`

**RX** `**4* K6ABr  DE N4DEF N4DEet N4DEF`

*Callsign only — repeated three times through light garble, readable. No exchange yet: this is normal, not an error.*

**TX** `N4DEF de K6ABC K6ABC · Pse copy 2A 2A STX STX · de K6ABC BK`

**RX** `**** K6ABC · QSL  PSE COPY 1D  1D  NFL DE N4DEe  K`

*Their QSL and exchange arrive together in the second over.*

**TX** `QSL 1D NFL TU N4DEF de K6ABC GL FD sk sk`

✓ **LOGGED** — the state machine tolerates the exchange arriving one over late; the callsign was confirmed before anything was logged.

### Example 3 · K0GHI — no copy, abandoned correctly

NOT LOGGED

| | |
|---|---|
| TX | CQ FD CQ FD de K6ABC K6ABC pse k |
| RX | 6ABC de K0GHI K0GH · pse cpy 1E UT 1E UT |
| TX | K0GHI de K6ABC K6ABC · Pse copy 2A 2A STX STX · de K6ABC BK |
| RX | nt z repeat no cpy |
| | *They didn't copy us — resend once, abbreviated (playbook, row 3).* |
| TX | K0GHI de K6ABC · 2A STX 2A STX · BK |
| RX | cptt t te e t eg |
| TX | K0GHI de K6ABC AGN pse k |
| RX | (silence — two listen windows) |
| TX | CQ FD CQ FD de K6ABC K6ABC pse k |

✗ **NOT LOGGED** — we never confirmed they copied our exchange. Rule 3 of the standard: an incomplete QSO is not a contact. Two overs spent, then back to work.

# Logging to N3FJP

The verified protocol, and every quirk we hit on the air.

### Preferred: one call

```
log → log_qso  {"call": "W7XYZ", "contest": "field_day",
                "exchange": {"class": "1D", "section": "AZ"}}
```

### Manual alternative

```
log → set_many  {"CALL": "W7XYZ", "CLASS": "1D", "SECTION": "AZ"}
log → enter
```

> SUCCESS IS LOGGED: TRUE — TRUST NOTHING ELSE
>
> Both paths return a `logged` boolean derived from the QSO-count delta and/or an ENTEREVENT push. `records_added` can report `0` for a QSO that *was* written — N3FJP networked (master-table) mode commits asynchronously. Retry only when `logged` is false.

### Quirks, observed live

- `set_many → calltab` can trigger an ENTEREVENT directly (the band/mode/dupe check fires on callsign entry). If it does, the QSO is logged — do not send another `enter`.
- `calltab` also populates COUNTRY/STATE/section lookups — useful for checking that the section you copied is plausible for the callsign.
- A `CALLTABDUPEEVENT` means the call is already in the log for this band/mode — usually a sign you (or a retry) already logged it.
- Exchange fields belong in the `exchange` dict of `log_qso` — passing them as top-level parameters silently logs nothing.

CHAPTER I · REFERENCE

# About & Provenance

These skills were not designed on paper. They are the distillate of one real autonomous contest session.

At ARRL Field Day 2026, a class-2A club station ran 20-meter BPSK31 with an agent operating fldigi and N3FJP through `fldigi-mcp` and `n3fjp-mcp` — calling CQ, working callers, and logging contacts, with the human operator supervising and able to stop transmission at any moment. Every rule, timing, and special case in this guide comes from that session's after-action report, `n3fjp-mcp/docs/LESSONS-FIELD-DAY-2026.md`.

| RESOURCE | WHERE |
|---|---|
| **fldigi-mcp** | `github.com/sbrunner-atx/fldigi-mcp · PyPI: fldigi-mcp` |
| **n3fjp-mcp** | `github.com/sbrunner-atx/n3fjp-mcp · PyPI: n3fjp-mcp` |
| **Skill sources** | `<repo>/skills/<name>/SKILL.md` |
| **Skill downloads** | ae5vg.com Downloads, one file per skill, under each server |
| **This guide's sources** | `<repo>/docs/brand/ (HTML + CSS, WeasyPrint)` |
| **After-action report** | `n3fjp-mcp/docs/LESSONS-FIELD-DAY-2026.md` |

Corrections and additions are welcome — open an issue on either repository.

> 73
>
> This is private, personal work by Stefan Brunner, AE5VG — built for amateur radio operators. Not affiliated with fldigi, N3FJP, or any employer. GPL-3.0-or-later, like fldigi itself. GL es 73 de AE5VG sk.